LayoutSpec3D: 3D Scene Generation with Structured Scene Descriptions and Asset-Aware Relation Correction

***Abstract*—Text-driven 3D scene generation needs to turn object requirements, spatial relations, and static geometric constraints into usable layouts. Data-driven and large-model methods often yield omissions, unmet relations, collisions, and boundary violations after numerical layouts are grounded in 3D assets. We propose LayoutSpec3D, a configuration-driven framework for scenes with explicit object requirements, spatial relations, and boundary conditions and sufficient asset coverage. It converts text, scene dimensions, boundaries, and domain configurations into a structured scene description, then builds a 3D asset record from each selected asset's geometry to rectify support, collision, and boundary constraints. The framework organizes outdoor scenes when requirements are specified and assets are available. LayoutSpec3D-FTCE (LayoutSpec3D Functional Topology Controlled Evaluation) fixes prompts and scene metadata, maps outputs to a common canonical scene, and applies shared final-scene checks. LayoutSpec3D delivers more complete and relation-consistent scene layouts while reducing geometric conflicts, including collisions, boundary violations, support errors, and floating objects. It further preserves intended spatial relations with minimal layout displacement and efficient generation.**

***Index Terms*—3D scene generation, text-to-3D, structured scene description, asset-aware layout, geometric constraint rectification.**

I. Introduction

3D scene generation constructs layouts from natural-language descriptions, scene dimensions, and boundaries for embodied intelligence, virtual environments, and digital content. Dense or function-oriented scenes need to meet object, spatial, and boundary requirements before rendering, interaction, or simulation. Across 3D environments, asset support, collision avoidance, and usable space determine whether an instantiated layout remains usable [1]–[4]. They also extend to outdoor scenes when the required assets are available.

Text-to-3D generation needs to instantiate language-derived counts, placement cues, and relations on specific objects. For example, wall attachment, tabletop placement, facing, adjacency, and passage clearance require object roles, support, orientation, distance, and boundaries. Their interpretation depends on the identities and roles of the participating instances rather than on categories alone. Prior work on functional and relation-based scene representations shows that object sets alone do not fully express functional organization [5]–[9]. Preserving the link from a textual requirement to its participating instances helps distinguish an omitted object from an unmet relation after layout generation. 3D-asset layout generation therefore needs an explicit correspondence among textual intent, object instances, and spatial relations.

An instance-level representation is especially important when a scene includes repeated categories. Two chairs may share a category yet participate in different support, facing, or clearance relations. Associating each relation with its subject and object instances prevents an adjustment intended for one object from being evaluated against another. Work on 3D scene graphs similarly links object semantics, 3D space, and inter-object relations within one representation [10].

Existing work mainly follows two paths. Data-driven methods learn joint distributions of objects and space from large scene collections [11]–[17], but their outputs are shaped by training data, object categories, and asset distributions. Language and vision–language models convert instructions into structured representations or numerical layouts [18]–[22]; these plans need consistency with final asset geometry. The handoff from a plan to selected assets is therefore more than a format conversion: it determines whether the intended instances and relations remain feasible in the final layout.

Structured scene representations, object identifiers, and executable descriptions make objects and relations checkable [23]–[26]. However, when requirements, initial layouts, and asset selection are maintained in separate stages, an omission, an unmet relation, and an asset-induced violation are difficult to attribute consistently. When the dimensions used to form an initial layout differ from those of the selected final assets, collisions, boundary violations, or relation drift arise; moving objects then disrupts functional relations [21], [27]–[29]. Planning, asset integration, and rectification need a shared instance-level representation.

The difference between a planning proxy and an instantiated asset also affects correction and evaluation. A layout may satisfy category-level placement terms while violating the footprint, support geometry, or boundary clearance of the selected 3D asset. Recording the final geometry with the related instances makes the source of a failure observable: it may arise from the request, the initial layout, the selected asset, or a later rectification step.

This mismatch also involves orientation and support rather than only scale. An initial proxy may locate a chair beside a table using nominal width and depth, while the selected OBJ geometry may have a rotated footprint, an offset origin, or a support face that changes its occupied region and contact state. A wall-mounted or tabletop instance has the same issue: a position that looks plausible at the category level may not preserve its intended attachment once the final asset axes and dimensions are applied. Retaining the instance identifier across the planned layout and selected geometry keeps this discrepancy attributable to one requirement, one object, and

one asset, rather than treating it as a generic error in the scene.

A reliable final-scene check therefore needs to read requirements and geometry together. It needs to determine whether required instances are present, whether named relations remain satisfied after assets are placed, and whether collision, boundary, and support constraints hold for the same instances. These questions require a common canonical scene instead of independent outputs from each method. The canonical scene preserves final poses and asset-derived geometry under a common representation, so planning omissions, relation drift, and static violations remain distinguishable during assessment. This separation also makes a later correction interpretable: its relation-preservation and displacement measures refer to the objects that participated in the original request, not only to category-level layout changes.

Methods differ in prompts, randomness, asset sources, and output formats, which confounds raw comparisons [18], [21], [22]. For example, SceneEval jointly evaluates explicit object and relation requirements and spatial plausibility, including support and collision [30]. A common final-scene representation makes comparisons less dependent on native output formats and helps distinguish planning, asset integration, and geometric correction. 3D-asset layouts benefit from this representation, which separates object completeness, relation satisfaction, and static geometric validity.

These gaps motivate LayoutSpec3D, which targets scenes with explicit object requirements, spatial relations, boundary conditions, and sufficient asset coverage. Its scene description records object requirements and critical relations, and the 3D asset record stores the selected asset's dimensions, coordinate conventions, and geometry. Static rectification checks support, collisions, boundaries, and relation drift. With the required configuration and assets, the framework organizes indoor and outdoor scenes and records relation preservation, rectification displacement, and runtime.

The main contributions are as follows:

- We propose LayoutSpec3D, an extensible framework for structured 3D scene generation. It applies to scenes with explicitly defined object requirements, spatial relations, and boundary conditions. After domain configuration and asset mapping, it organizes indoor and outdoor scenes with the corresponding assets.
- We establish a closed loop that combines a structured scene description with static rectification of instance-level 3D assets. The scene description explicitly records object requirements and critical spatial relations for subsequent checks. Using 3D asset records, the rectification module handles support alignment, collision resolution, boundary constraints, and relation recovery while limiting disruption to the original functional relations.
- We define LayoutSpec3D-FTCE (LayoutSpec3D Functional Topology Controlled Evaluation), a unified evaluation procedure for final scenes. Across different scene and functional configurations, it fixes prompts, random seeds, and scene metadata, converts outputs from each method into a common canonical scene, and reports object completeness, relation satisfaction, and static geometric validity under identical criteria applied after generation.

II. Related Work

A. Data-driven 3D scene generation, primarily for indoor scenes

Data-driven 3D scene generation learns object co-occurrence, layout, and semantic priors from large scene collections. 3D-FRONT and 3D-FUTURE provide indoor layouts, furniture assets, and semantic annotations [11], [12]. GRAINS, Fast and Flexible Indoor Scene Synthesis, ATISS, and DiffuScene use recursive, convolutional, autoregressive, and diffusion models [13]–[16]. Later work adds relation priors, asset attributes, and explicit constraints [5]–[7], [31], [32]. Relation-aware priors encode common placement patterns, yet they may not bind each declared relation to the particular 3D asset used in the final scene. The geometry used to form a generated arrangement is often derived from category-level examples or learned distributions. Once a requested role is bound to a specific 3D asset, its footprint and support geometry may differ from that approximation. A category-level plan therefore does not identify the selected asset for a requested role or establish whether it fits the layout. LayoutSpec3D instead uses domain configurations, asset mapping, and an evaluable intermediate representation without learning a target-domain distribution.

B. Language-model layout planning and structured scene representations

Language and vision–language models translate textual intent into explicit plans. LayoutGPT uses CSS-like representations and in-context examples; DirectLayout separates top-view planning, 3D completion, and asset alignment; TreeSearchGen combines multi-level scene representations with global and local tree search [18], [21], [22]. Scene Language, Holodeck, AnyHome, InstructScene, LayoutVLM, SceneCraft, and Planner3D use scene languages, open vocabularies, semantic graphs, layout optimization, Blender scripts, or graph structures [6], [19], [20], [26], [28], [33], [34]. These representations make textual requirements executable, but category-level plans lack selected-asset dimensions, coordinate conventions, and support surfaces. A scene language or graph exposes which objects and relations are requested, whereas final asset geometry determines whether a numerical placement remains feasible. Retaining the same instance identity across these stages links a later conflict to its originating requirement instead of treating it as an unstructured coordinate error. Differences in final asset geometry therefore invalidate planned placements or relations. LayoutSpec3D retains an instance-identified scene description through asset mapping and static rectification.

C. Asset-aware geometric validity and physical scene construction

3D assets differ from category-level dimensions, so numerical layouts need geometric checks. PhyScene checks collisions, boundaries, and accessibility; DirectLayout and LayoutVLM use asset geometry for collision, boundary, and local-relation handling [21], [27], [28]. Research on instance

segmentation, pose estimation, semantic metric mapping, and constrained registration relies on instance identity and metric geometry [35]–[38]. Physics-constrained pose refinement models support relations during geometric adjustment, while collision detection motivates explicit conflict checks [39], [40]. Asset-aware checks also distinguish valid support contact from invalid intersection; otherwise, an object placed on a table is incorrectly classified as colliding. This distinction depends on the selected asset's orientation, support type, and footprint rather than only on an axis-aligned planning volume. When a final asset is rotated, its effective occupied area further depends on its yaw. Recent scene-aware asset retrieval work considers global coherence when selecting 3D assets for generated scenes [41], but it does not explicitly verify support contact, collision freedom, or boundary compliance after placement. The selected asset's geometry thus affects retrieval coherence and the definition of a valid final layout. SceneSmith and SceneWeaver include asset validation and scene construction [29], [42]. LayoutSpec3D focuses on post-instantiation static validity: the 3D asset record stores asset properties, and staged rectification addresses support, collision, boundaries, and relation preservation while recording preservation, displacement, and runtime.

D. Evaluation of text-to-3D scenes

Text-to-3D studies usually use their own scene collections, prompts, assets, and pipelines. DirectLayout compares indoor scene types, TreeSearchGen introduces 3DTindo-Bench, and SceneSmith studies house-scale realism, layout consistency, and runtime [21], [22], [29]. These differences make attribution to planning, asset mapping, or post-processing difficult. Evaluation also depends on the form in which final scenes are compared. Without a common final-scene representation, a method's native object encoding or later processing may influence the measured outcome independently of its layout decisions. A fixed converted scene makes semantic requirements, geometric validity, and relation preservation observable under the same definitions. To mitigate this confounding, SceneEval evaluates explicit object and relation requirements together with spatial plausibility, including support and collision [30]. LayoutSpec3D-FTCE is a unified evaluation procedure for final scenes, not a benchmark: it fixes prompts, random seeds, and scene metadata, maps outputs to a canonical scene, and applies common geometric and relational checks.

III. Method

A. Problem definition

Given a text scene request, dimensions, boundaries, a domain configuration, and available assets, LayoutSpec3D treats text-to-3D generation as structured construction and instantiation. The domain configuration defines the object vocabulary, quantity requirements, functional zones, relation templates, and placement types. During language-model compilation, the request yields a structured scene description and an initial layout with shared instance identifiers. The description records declarative requirements, while the layout sets each instance's position, scale, yaw, and anchor. The system records input, planning, asset-integration, and rectification states, tracing requirements, relations, and selected assets through the scene description and 3D asset record. This shared representation is consistent with prior scene representations that retain structured object relations and instance identifiers [23]–[25].

We formalize the structured scene description as

$$\mathcal{S} = (D, V, Z, R, O, E, B, A, \Omega), \tag{1}$$

where D, V, Z, R, O, and E denote the domain, structural variant, functional zones, object categories and quantities, identified instances, and relations; B, A, and Ω specify dimensions and boundaries, placement anchors, and orientation conventions.

For scenes with explicit object requirements, spatial relations, boundaries, and sufficient asset coverage, let $\mathbf{q}^{(0)}$ be the initial layout produced with $\mathcal{S}$, and let G be the asset geometry and mapping information. Static rectification produces

$$\mathbf{q}^{*} = \mathrm{Rectify}(\mathbf{q}^{(0)}; \mathcal{S}, G). \tag{2}$$

Here, $\mathbf{q}$ contains instance position, orientation, and support height; $\mathrm{Rectify}(\cdot)$ is the relation-preserving static geometric rectification operator; $\mathcal{S}$ specifies types, quantities, zones, boundaries, anchors, orientation conventions, and relations; and G provides final-asset dimensions, coordinate axes, support types, and footprints. Rectification handles static support, collision, boundaries, and relation preservation; the interface accommodates later dynamics, accessibility, and fine-mesh checks.

B. LayoutSpec3D overview

Direct text-to-layout conversion risks conflating textual intent with later geometric errors. CSS-like layouts, scene languages, graphs, executable scene code, and other structured descriptions represent objects and relations [7], [18], [20], [23], [26], [34], [43], but do not isolate asset mismatches or geometric conflicts without retained instance identities and final geometry. LayoutSpec3D instead retains both in a structured description shared by asset mapping and rectification.

LayoutSpec3D-FTCE is a unified evaluation protocol, not a generation stage: it converts final outputs from all compared methods to a canonical scene and applies common checks. LayoutSpec3D uses domain configurations, instance-level assets, and static constraints to construct the description, map assets, rectify geometry, and recheck relations. Its recorded stages separate planning, asset-geometry, and rectification changes.

C. Building a structured scene description

Planning representations do not replace final assets. Bounding boxes, CasaGPT's cuboid primitives, and VoxScene's discrete occupancy representation support planning [44], [45], but do not provide a target OBJ asset's dimensions, coordinate conventions, support face, or rotated footprint. The structured scene description is therefore the interface between language-model compilation and later geometric operations.

The description separates declarative scene requirements from instance-level placement fields. Category and quantity

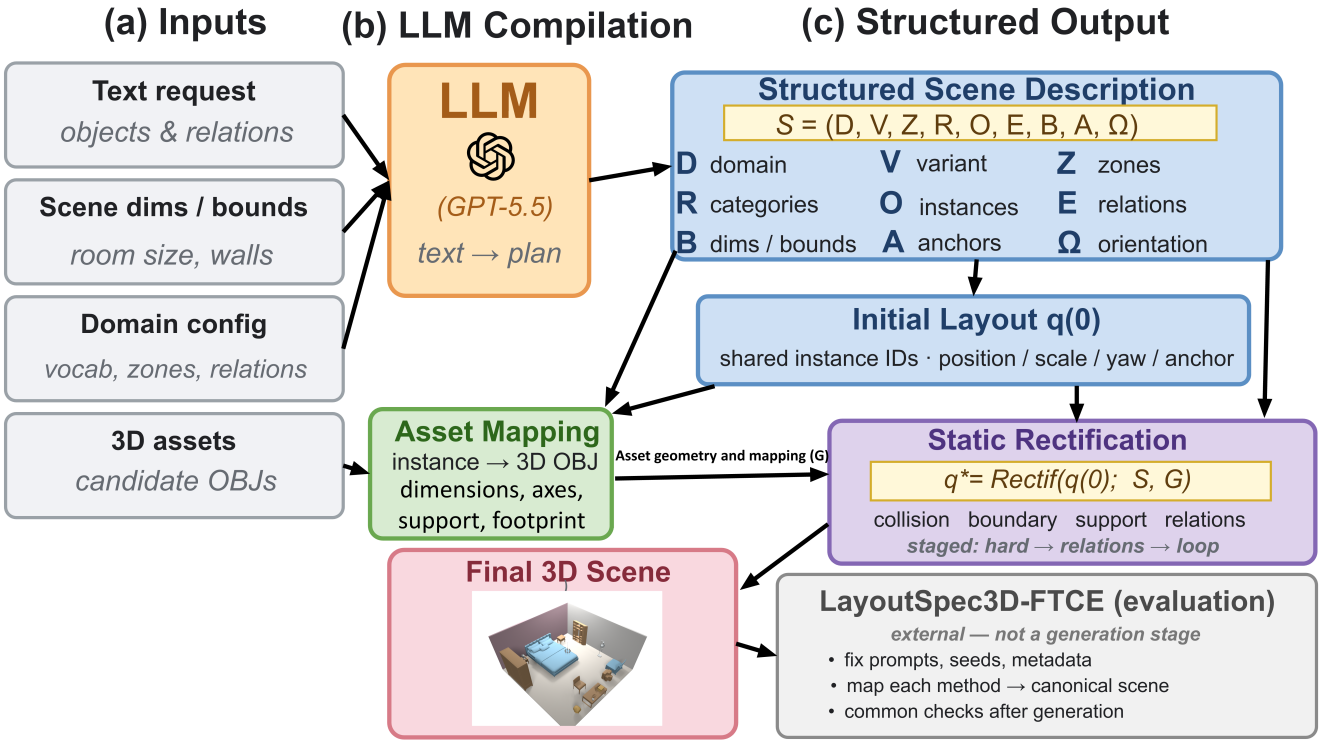


Fig. 1. Overview of LayoutSpec3D.

requirements state what the scene needs, while each identified instance carries its target scale, placement anchor, and references to the relations in which it participates. A relation is associated with particular subject and object instances rather than with a category pair. This distinction is relevant when several instances share a category but occupy different functional roles. For example, two instances from the same category may have different support targets or orientation relations, even when their category and target scale are identical.

During asset mapping, the instance identifier retains the link from the compiled request to the selected 3D asset. The same identifier remains in the initial layout, 3D asset record, and rectified layout, so an unmet relation remains connected to its source object and selected asset. The retained fields also localize later exceptions. An absent required instance, an unsatisfied declared relation, and an unsuitable asset binding are different cases in the record rather than one undifferentiated layout error. The description does not decide the final occupancy of an asset; it supplies the object, relation, anchor, and boundary constraints against which the final geometry is checked.

At the boundary between planning and geometry, the initial layout supplies pose values for the identified instances, while the structured scene description supplies the requirements that those values are expected to satisfy. Asset mapping reads both views together: position and yaw establish where an asset is placed, and relation references determine which support, alignment, attachment, or clearance checks apply. This split keeps planning information available without treating it as final geometry.

D. Instance-level asset mapping and geometry integration

After the scene description determines requirements and relations, LayoutSpec3D maps each abstract object to a 3D asset for rectification. Each instance is bound to an available asset from its category-specific candidate set. The selection stays fixed, even when same-category instances use different assets. Binding therefore preserves the role of each abstract object instead of replacing all instances in a category with one representative geometry.

Each binding creates a 3D asset record that links the instance identifier to the selected source. The record stores the semantic category, path, source dimensions, width–depth–height correspondence, up and forward conventions, target dimensions, scaling and origin policies, support type, and rotated footprint. Source dimensions and the width–depth–height correspondence establish how the source axes map to the target dimensions. The up and forward conventions establish the orientation used by placement anchors and yaw, while the support type identifies the contact condition relevant to the instance. Taken together, these fields make the selected asset's geometry available to the later support, collision, and boundary checks.

The OBJ check verifies coordinate conventions and support faces before the record enters rectification. It checks whether the selected source provides a compatible vertical direction, forward direction, and support surface for the role specified in the scene description. If the selected source lacks vertical or forward direction, a compatible fallback is used; otherwise, the binding is marked failed. The same record then supplies

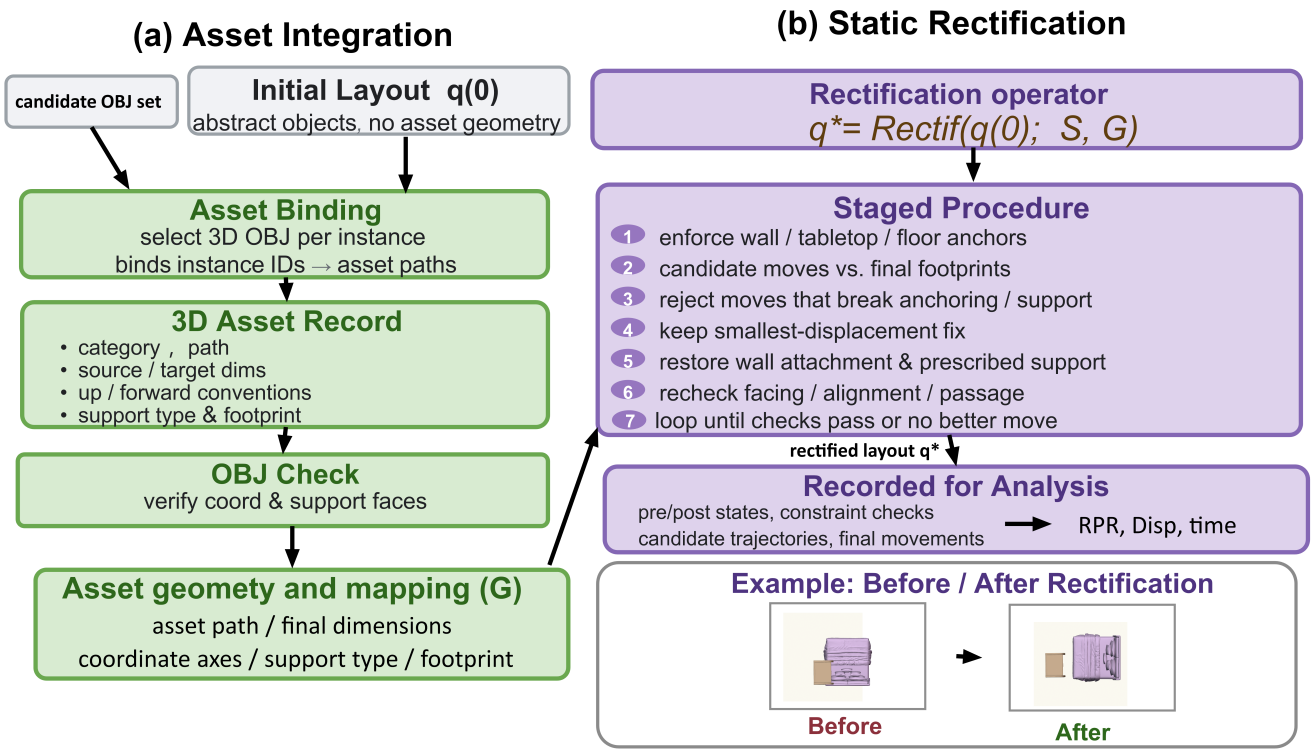


Fig. 2. Framework for instance-level asset integration and relation-preserving static geometric rectification. Initial-layout instances are bound to candidate 3D OBJ assets; the resulting records and OBJ checks supply asset geometry and mapping information for collision, boundary, and support checks, staged adjustment, and relation rechecking. Pre/post states, checks, trajectories, and movements are recorded for analysis.

retrieval, dimension, and collision information [29]. Keeping this decision with the instance prevents a later geometry check from losing the connection to the asset that caused it.

After mapping, final asset dimensions and yaw define an oriented footprint, collision body, and boundary-check object for every instance. Rectification therefore uses 3D-asset occupancy rather than planning boxes. The footprint is updated with the instance pose during candidate adjustment, so collision and boundary checks remain tied to the geometry actually placed in the scene. The final record retains asset identity and geometry, distinguishing a missing object, a mapping error, and a rectification failure in the resulting scene.

E. Relation-preserving static geometric rectification

After asset instantiation, LayoutSpec3D applies staged, relation-preserving static geometric rectification. It removes geometric conflicts, retains functional relations, and limits unnecessary displacement:

$$\mathbf{q}^* \approx \arg \min_{\mathbf{q} \in \mathcal{F}(\mathcal{S},G)} \left[\lambda_c E_c + \lambda_b E_b + \lambda_s E_s + \lambda_r E_r + \lambda_m E_m\right]. \tag{3}$$

Here, $\mathcal{F}(\mathcal{S}, G)$ contains states that satisfy the scene description and asset-geometry constraints. E_c, E_b, E_s, E_r, and E_m are the costs of collision, boundary violation, support violation, relation drift, and displacement; λ_c, λ_b, λ_s, λ_r, and $\lambda_m \geq 0$ set their relative priorities. The $\approx$ characterizes the staged objective rather than a global optimum. The terms distinguish geometric invalidity from relation drift and unnecessary motion, so the rectification objective does not treat all layout changes as equivalent. The system handles hard constraints before restoring relations.

Rectification is staged by relation type and asset anchor. The first stage establishes conditions that define a feasible attachment or support state: wall-attached instances retain their wall anchors, tabletop instances are aligned with their designated support surfaces, and floor instances are brought into floor contact. A local candidate move is rejected when it breaks one of these established anchor or support conditions. This ordering prevents a collision reduction from being retained when it removes the support or attachment required by the structured scene description.

The next stage evaluates local candidate moves against final footprints, collision bodies, and scene boundaries. Each candidate is checked using the geometry from the selected 3D asset rather than the proxy geometry used during initial planning. Among candidates that retain the required anchor and support state, the procedure retains the smallest-displacement move that reduces a collision or boundary violation. When a pose changes, the corresponding footprint and local geometric checks are updated before the next decision. This process limits changes to the original layout while giving occupied area and boundary compliance priority over a coordinate adjustment alone.

Once the hard geometric conditions are satisfied, rectification rechecks facing, alignment, wall attachment, support,

and passage relations. Relation rechecking is separate from the initial language-model compilation: it verifies whether the final asset geometry and accepted local moves still satisfy the relation references in the structured scene description. The process ends when all checks pass or no candidate improves the current violation. Unresolved cases are retained in the record rather than being forced into an unsupported correction [44]–[46].

Pre- and post-modification states, constraint checks, candidate trajectories, and final movements are saved for ablations and relation-preservation, displacement, and runtime measures. The paired states identify which previously satisfied relations remain satisfied after rectification for RPR, while matched object positions provide the input for the normalized displacement measure. The traces retain each accepted movement alongside the checks that motivated it, so a correction remains associated with the collision, boundary, or support issue it addressed. The same staged record distinguishes planning, asset-integration, and rectification changes when end-to-end runtime is reported.

IV. Experiments

We evaluate LayoutSpec3D through baseline comparisons, module ablations, and scope examples. The experiments examine object and relation satisfaction, geometric validity on jointly supported scenes, each core module, and configurations beyond the standard indoor comparison.

A. Experimental setup

To standardize language-model-dependent generation and reasoning, we use GPT-5.5 at each method's language-model call sites. For LayoutSpec3D, GPT-5.5 only compiles the text request into an initial structured scene description and layout plan; asset mapping, geometry integration, static rectification, canonical-scene conversion, and post-generation evaluation do not invoke a language model. The frozen LayoutSpec3D-FTCE configuration contains 105 prompts at three granularities and three fixed random seeds, giving each method up to 315 prompt–seed combinations. Comparisons share configurations, prompts, and seeds; outputs are converted to a canonical scene and checked after generation. Domains unsupported by a native implementation are excluded [18], [21], [22].

Quantitative comparisons use jointly supported indoor scenes, with shared dimensions, object requirements, relation descriptions, and asset interfaces. Common checks measure semantic requirements, geometric validity, relation preservation, displacement, and runtime. Values are percentages except Eff. Sem. Richness, Disp., and Time, following SceneEval and PhyScene [27], [30].

1) Evaluation metrics: Let $\mathcal{I}_m$ be the records for method m that produce an output and complete canonical-scene conversion. For record i, O_i is the final object set, and C_i and B_i contain objects in invalid collisions and out-of-bounds cases. An invalid collision is a non-support geometric intersection. We report semantic satisfaction, static geometric validity, and rectification preservation separately [27], [30].

Semantic satisfaction. For category c in the scene requirements, $n_i(c)$ and $n_i^{\mathrm{req}}(c)$ denote the generated count and the required minimum count, respectively. Object recall and relation satisfaction are defined as

$$\text{O-Recall}_m = \frac{1}{|\mathcal{I}_m|} \sum_{i \in \mathcal{I}_m} \frac{\sum_c \min\{n_i(c), n_i^{\mathrm{req}}(c)\}}{\sum_c n_i^{\mathrm{req}}(c)}, \tag{4}$$

$$\text{Relation Sat.}_m = \frac{1}{|\mathcal{I}_{S,m}|} \sum_{i \in \mathcal{I}_{S,m}} \frac{\sum_{r \in R_i} \chi(r \text{ is satisfied})}{|R_i|}, \tag{5}$$

Here, $\chi(\cdot)$ is the indicator function, $\mathcal{I}_{S,m} \subseteq \mathcal{I}_m$ contains records with declared relations, and R_i is the relation set of record i. A relation is satisfied when all generated subject instances meet its wall-attachment, proximity, alignment, orientation, or support condition. Effective semantic richness counts unique categories among geometrically valid objects:

$$\text{ESR}_m = \frac{1}{|\mathcal{I}_m|} \sum_{i \in \mathcal{I}_m} \left| \{ \mathrm{cat}(o) \mid o \in O_i \setminus (C_i \cup B_i) \} \right|. \tag{6}$$

where $\mathrm{cat}(o)$ denotes the semantic category of object o.

Static geometric validity. Col-obj and OOB-obj are mean within-scene proportions of invalid collisions and out-of-bounds objects; Col-scene and OOB-scene are record proportions with at least one case. Support-obj and Float-obj are prescribed-support violation and floating-object rates. Collision checks use final-asset footprints and vertical occupancy, and boundary checks use footprint and vertical extent. Valid support contacts are not collisions.

Support and floating status are computed from final canonical-scene oriented bounding boxes. For object o, let z_o and h_o be its center height and height, with bottom and top heights $z_o^- = z_o - h_o/2$ and $z_o^+ = z_o + h_o/2$; F_o is its rotated xy footprint. For supported object s and candidate carrier c, the contact gap and projected coverage ratio are

$$g(s,c) = \left| z_s^- - z_c^+ \right|, \qquad \rho(s,c) = \frac{\mathrm{Area}(F_s \cap F_c)}{\mathrm{Area}(F_s)}. \tag{7}$$

where $\mathrm{Area}(\cdot)$ is horizontal-plane area; $\epsilon_z = 0.05\,\mathrm{m}$ and $\tau_{\mathrm{cover}} = 0.50$. Floor and valid inter-object contact are

$$\begin{aligned} \mathrm{Floor}(s) &= \chi(|z_s^-| \le \epsilon_z), \\ \mathrm{Contact}(s,c) &= \chi(g(s,c) \le \epsilon_z \ \wedge \ \rho(s,c) \ge \tau_{\mathrm{cover}}). \end{aligned} \tag{8}$$

Support and floating. Each test case has floor or designated-object support targets. Let $T_i = T_i^{\mathrm{floor}} \cup T_i^{\mathrm{object}}$ be the targets in record i, s_t the target subject, c_t the declared carrier of an object-support target, and $q_i(t)$ its satisfaction indicator:

$$q_i(t) = \begin{cases} \mathrm{Floor}(s_t), & t \in T_i^{\mathrm{floor}},\ s_t \text{ exists}, \\ \mathrm{Contact}(s_t, c_t), & t \in T_i^{\mathrm{object}},\ s_t, c_t \text{ exist}, \\ 0, & \text{otherwise}. \end{cases} \tag{9}$$

The object-level support error rate is defined as

$$\text{Support-obj}_m = \frac{1}{|\mathcal{I}_{T,m}|} \sum_{i \in \mathcal{I}_{T,m}} \frac{1}{|T_i|} \sum_{t \in T_i} (1 - q_i(t)), \tag{10}$$

Here, $\mathcal{I}_{T,m} \subseteq \mathcal{I}_m$ contains records with nonempty T_i. A missing required object or carrier, floor separation, an excessive gap, or insufficient coverage is a support violation.

TABLE I
QUANTITATIVE COMPARISON ON JOINTLY SUPPORTED STANDARD SCENES. VALUES ARE PERCENTAGES EXCEPT FOR EFF. SEM. RICHNESS, DISP., AND TIME.

Method	Eff. Sem. Richness ↑	O-Recall ↑	Relation Sat. ↑	Col-obj ↓	Col-scene ↓	OOB-obj ↓	OOB-scene ↓	Support-obj ↓	Float-obj ↓	RPR ↑	Disp. ↓	Time (s) ↓
DirectLayout	2.14	68.94	45.15	27.76	49.51	36.11	54.37	2.45	2.39	–	–	216.38
LayoutGPT	1.53	88.25	15.08	35.61	62.43	50.94	77.25	1.14	3.24	88.09	0.07	60.66
TreeSearchGen	0.64	44.09	59.32	17.01	32.28	37.54	36.61	1.95	2.84	–	–	569.66
LayoutSpec3D	**2.34**	**95.76**	**66.21**	**9.09**	**23.39**	**15.92**	**14.21**	**0.92**	**1.17**	**96.48**	**0.02**	**48.57**

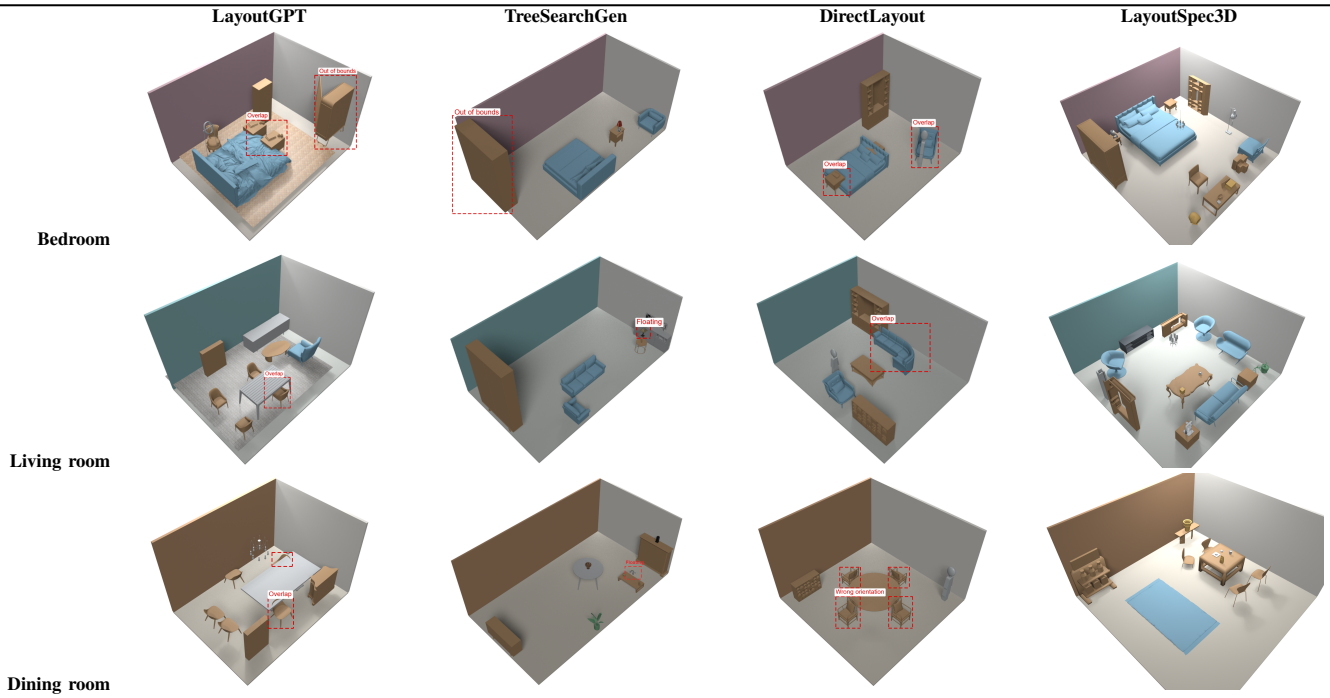


Fig. 3. Qualitative comparison of bedroom, living-room, and dining-room scenes. Each row is generated under identical object requirements, spatial relations, and boundary conditions.

Float-obj tests physical support rather than designated-carrier use. Let O_i^{chk} contain objects in record i eligible for floor or object support checks, excluding floor decorations and objects with ceiling- or wall-attachment contracts. For $s \in O_i^{\mathrm{chk}}$, define

$$p_i(s) = \chi\big(\mathrm{Floor}(s) \\ \vee \ \exists c \in O_i^{\mathrm{chk}} \setminus \{s\} : \mathrm{Contact}(s,c)\big), \tag{11}$$

The floating-object error rate is defined as

$$\text{Float-obj}_m = \frac{1}{|\mathcal{I}_{F,m}|} \sum_{i \in \mathcal{I}_{F,m}} \frac{1}{|O_i^{\mathrm{chk}}|} \sum_{s \in O_i^{\mathrm{chk}}} (1 - p_i(s)), \tag{12}$$

Here, $\mathcal{I}_{F,m} \subseteq \mathcal{I}_m$ contains records with nonempty O_i^{chk}. An incorrect carrier causes a Support-obj failure, but an object with physical support is not floating. Thus the metrics distinguish prescribed-support errors from actual lack of support.

Rectification preservation and process measures. Let R_i^{raw} be the index set of relations satisfied before rectification and R_i^{final} the corresponding set satisfied after rectification. The relation preservation rate is

$$\mathrm{RPR}_m = \frac{1}{|\mathcal{I}_{R,m}|} \sum_{i \in \mathcal{I}_{R,m}} \frac{|R_i^{\mathrm{raw}} \cap R_i^{\mathrm{final}}|}{|R_i^{\mathrm{raw}}|}, \tag{13}$$

Here, $\mathcal{I}_{R,m} \subseteq \mathcal{I}_m$ retains records with relations satisfied before rectification. For record i, M_i is the set of post-/pre-rectification object pairs from greedy category matching. In $(a,b) \in M_i$, a is the post-rectification object and b its matched pre-rectification object; p_a^{final} and p_b^{raw} are their horizontal center positions. Layout displacement is normalized by the room diagonal:

$$\mathrm{Disp.}_m = \frac{1}{|\mathcal{I}_{D,m}|} \sum_{i \in \mathcal{I}_{D,m}} \frac{1}{|M_i|} \sum_{(a,b) \in M_i} \frac{\left\| p_a^{\mathrm{final}} - p_b^{\mathrm{raw}} \right\|_2}{\sqrt{L_i^2 + W_i^2}}. \tag{14}$$

Here, $\|\cdot\|_2$ is the two-dimensional Euclidean norm, $\mathcal{I}_{D,m}$ contains records with nonempty matching sets, and L_i and W_i are scene length and width. RPR and Disp. require paired layouts before and after rectification. Time (s) is mean end-to-end runtime per scene over $\mathcal{I}_m$.

Ablations use matched standard configurations with unchanged prompts, assets, random seeds, and checks on the final scene. They remove structured scene description construction, static rectification, or both on the same test subset.

B. Comparison with baselines

Table I summarizes jointly supported scenes. LayoutSpec3D leads in effective semantic richness (2.34), object recall

TABLE II

ABLATION RESULTS FOR THE CORE MODULES. SCENE DESC. DENOTES STRUCTURED SCENE DESCRIPTION CONSTRUCTION AND RECT. DENOTES RELATION-PRESERVING STATIC GEOMETRIC RECTIFICATION. ✓INDICATES A RETAINED MODULE AND – A REMOVED MODULE. ALL VALUES ARE PERCENTAGES.

Variant	Scene desc.	Rect.	O-Recall ↑	Relation Sat. ↑	Col-obj ↓	Col-scene ↓	OOB-obj ↓	OOB-scene ↓	Support-obj ↓	Float-obj ↓	RPR ↑
Full LayoutSpec3D	✓	✓	95.76	66.21	9.09	23.39	15.92	14.21	0.92	1.17	96.48
w/o Scene desc.	–	✓	67.36	55.25	11.53	25.08	16.84	15.00	1.45	2.25	48.04
w/o Rect.	✓	–	86.88	49.46	28.08	50.14	32.31	39.05	3.02	4.36	66.05
w/o Both	–	–	62.35	46.96	34.72	58.29	39.98	47.67	3.19	5.11	43.33

(95.76%), and relation satisfaction (66.21%), and has the lowest collision, out-of-bounds, prescribed-support, and floating rates. The scene description makes object and relation constraints explicit, while rectification checks geometric validity from 3D-asset footprints.

LayoutSpec3D attains 96.48% RPR with 0.02 displacement, compared with 88.09% and 0.07 for LayoutGPT. Its 48.57 s runtime is below LayoutGPT (60.66 s), DirectLayout (216.38 s), and TreeSearchGen (569.66 s). These are shared-environment measurements, not absolute software or hardware comparisons.

LayoutGPT reaches 88.25% object recall but 15.08% relation satisfaction, with 62.43% scene collisions and 77.25% scene out-of-bounds cases. Object coverage alone therefore does not guarantee agreement with 3D-asset occupancy. DirectLayout is also lower than LayoutSpec3D on semantic, object, relation, and geometric measures.

TreeSearchGen reaches 59.32% relation satisfaction, but its object recall is 44.09% and runtime 569.66 s. Under shared configurations, LayoutSpec3D has stronger object and relation results with lower geometric-error and runtime measures.

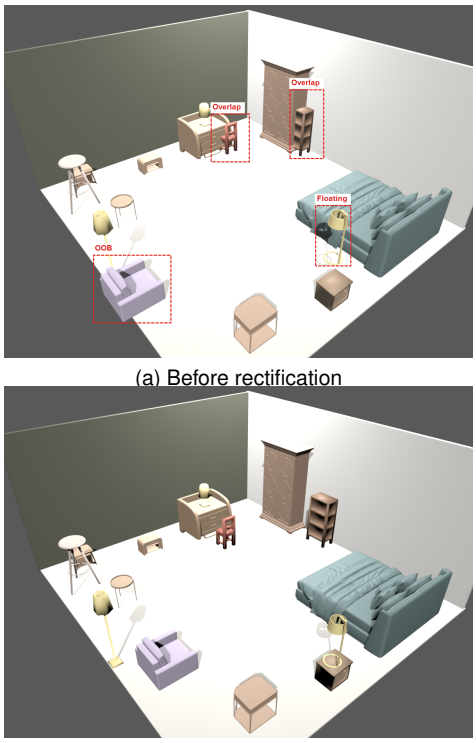
For a qualitative comparison, Fig. 3 compares the same scenes under identical requirements, spatial relations, and boundaries. The baseline examples include incomplete objects, offsets, or occupancy conflicts; LayoutSpec3D uses its scene description and instance-level footprints for rectification.

C. Ablation study

Table II isolates the two modules. Without structured scene description construction, object recall falls from 95.76% to 67.36%, relation satisfaction to 55.25%, and RPR to 48.04%; later rectification does not restore unstructured planning information. Without static rectification, object- and scene-collision rates rise to 28.08% and 50.14%, out-of-bounds rates to 32.31% and 39.05%, Support-obj to 3.02%, Float-obj to 4.36%, and RPR falls to 66.05%. Footprint, boundary, and support constraints benefit from explicit treatment after asset integration.

Removing both modules reduces object recall and relation satisfaction to 62.35% and 46.96%, with 3.19% Support-obj, 5.11% Float-obj, and 43.33% RPR. The single-module ablations separate the roles: without the scene description, later adjustment does not recover absent structured relations; without rectification, object recall remains 86.88%, but collision, out-of-bounds, support-violation, and floating-object rates increase. The scene description therefore converts textual requirements into checkable instance targets before rectification removes local conflicts.

Fig. 4 shows the rectification process. Object relations and asset instances remain traceable before and after correction, consistent with Table II.



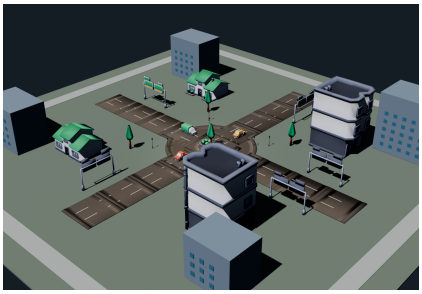
(a) Before rectification

(b) After rectification

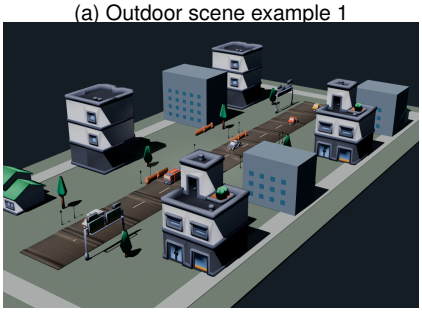
Fig. 4. Before-and-after comparison of static geometric rectification. Invalid overlap, out-of-bounds (OOB), and floating issues present before rectification are handled after rectification, while checkable object relations and asset-instance records are retained.

D. Qualitative results and scope

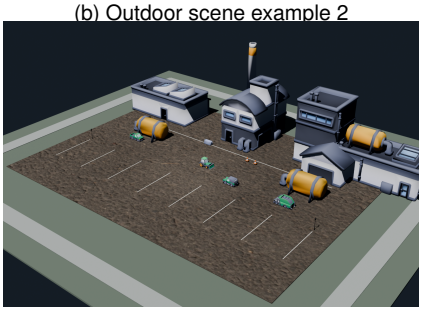
Beyond the standard indoor comparison, LayoutSpec3D also organizes scenes when object requirements, spatial relations, and boundaries are explicit and assets are available. Fig. 5 shows outdoor layouts with roads, buildings, and traffic objects. Comparable outdoor results are not included for DirectLayout, LayoutGPT, or TreeSearchGen under the shared protocol, so the examples illustrate scope rather than rankings against Table I.



(a) Outdoor scene example 1



(b) Outdoor scene example 2



(c) Outdoor scene example 3

Fig. 5. Outdoor-scene examples within the scope of LayoutSpec3D. The scenes are jointly determined by available assets, object requirements, spatial relations, and boundary configurations.

The indoor tables compare generation and geometric quality where all methods are supported; the outdoor examples show reuse of the same interface under changed configurations. The scene description supplies categories, relations, and boundaries; the 3D asset record supplies dimensions, coordinate conventions, and footprints; static rectification checks local geometry.

V. Conclusion

LayoutSpec3D grounds textual requirements, spatial relations, and static geometry in 3D assets. The scene description records objects and relations; the 3D asset record links each selected asset to its geometry; rectification handles support, collision, boundaries, and relation drift. On jointly supported indoor scenes, it improves object requirements, relation satisfaction, and geometric validity. Ablations distinguish the modules: the scene description provides checkable objects and relations, while rectification resolves post-instantiation occupancy conflicts. With explicit requirements, relations, boundaries, and asset coverage, LayoutSpec3D organizes indoor and outdoor scenes. The interface is extensible to dynamic interaction validation, accessibility analysis, and fine-grained geometric checks.

References

[1] E. Kolve, R. Mottaghi, W. Han *et al.*, "AI2-THOR: An interactive 3D environment for visual AI," *arXiv preprint arXiv:1712.05474*, 2017.

[2] B. Shen, F. Xia, C. Li *et al.*, "iGibson 1.0: A simulation environment for interactive tasks in large realistic scenes," in *IEEE/RSJ International Conference on Intelligent Robots and Systems*, DOI 10.1109/IROS51168.2021.9636667, pp. 7520–7527, 2021.

[3] M. Deitke, E. VanderBilt, A. Herrasti *et al.*, "ProcTHOR: Large-scale embodied AI using procedural generation," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 5982–5994, 2022, arXiv:2206.06994.

[4] S. Spellini, R. Chirico, M. Panato, M. Lora, and F. Fummi, "Virtual prototyping a production line using assume–guarantee contracts," *IEEE Transactions on Industrial Informatics*, vol. 17, DOI 10.1109/TII.2020.3038679, no. 9, pp. 6294–6302, 2021.

[5] H. Yi, C.-H. P. Huang, S. Tripathi *et al.*, "MIME: Human-aware 3D scene generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52729.2023.01246, pp. 12 965–12 976, 2023.

[6] C. Lin and Y. Mu, "InstructScene: Instruction-driven 3D indoor scene synthesis with semantic graph prior," in *International Conference on Learning Representations*, 2024, arXiv:2402.04717.

[7] G. Zhai, E. P. Örnek, S.-C. Wu *et al.*, "CommonScenes: Generating commonsense 3D indoor scenes with scene graph diffusion," in *Advances in Neural Information Processing Systems*, vol. 36, DOI 10.52202/075280-1307, pp. 30 026–30 038, 2023.

[8] A. M. Zanchettin, A. Casalino, L. Piroddi, and P. Rocco, "Prediction of human activity patterns for human–robot collaborative assembly tasks," *IEEE Transactions on Industrial Informatics*, vol. 15, DOI 10.1109/TII.2018.2882741, no. 7, pp. 3934–3942, 2019.

[9] J. Lv, R. Zhang, X. Li, S. Liu, T. Liu, Q. Zhang, and J. Bao, "A multimodality scene graph generation approach for robust human–robot collaborative assembly visual relationship representation," *IEEE Transactions on Industrial Informatics*, vol. 20, DOI 10.1109/TII.2023.3303964, no. 3, pp. 3242–3251, 2024.

[10] I. Armeni, Z.-Y. He, A. Zamir, J. Gwak, J. Malik, M. Fischer, and S. Savarese, "3D scene graph: A structure for unified semantics, 3D space, and camera," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, DOI 10.1109/ICCV.2019.00576, pp. 5663–5672, 2019.

[11] H. Fu, B. Cai, L. Gao *et al.*, "3D-FRONT: 3D furnished rooms with layouts and semantics," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, DOI 10.1109/ICCV48922.2021.01075, pp. 10 913–10 922, 2021.

[12] H. Fu, R. Jia, L. Gao *et al.*, "3D-FUTURE: 3D furniture shape with texture," *International Journal of Computer Vision*, vol. 129, DOI 10.1007/s11263-021-01534-z, no. 12, pp. 3313–3337, 2021.

[13] D. Paschalidou, A. Kar, M. Shugrina *et al.*, "ATISS: Autoregressive transformers for indoor scene synthesis," in *Advances in Neural Information Processing Systems*, vol. 34, pp. 12 013–12 026, 2021, arXiv:2110.03675.

[14] J. Tang, Y. Nie, L. Markhasin *et al.*, "DiffuScene: Denoising diffusion models for generative indoor scene synthesis," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52733.2024.01938, pp. 20 507–20 518, 2024.

[15] M. Li, A. G. Patil, K. Xu *et al.*, "GRAINS: Generative recursive autoencoders for indoor scenes," *ACM Transactions on Graphics*, vol. 38, DOI 10.1145/3303766, no. 2, pp. 1–16, 2019.

[16] D. Ritchie, K. Wang, and Y.-A. Lin, "Fast and flexible indoor scene synthesis via deep convolutional generative models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR.2019.00634, pp. 6175–6183, 2019.

[17] X. Wang, C. Yeshwanth, and M. Nießner, "SceneFormer: Indoor scene generation with transformers," in *International Conference on 3D Vision*, DOI 10.1109/3DV53792.2021.00021, pp. 106–115, 2021.

[18] W. Feng, W. Zhu, T.-J. Fu *et al.*, "LayoutGPT: Compositional visual planning and generation with large language models," in *Advances in Neural Information Processing Systems*, vol. 36, DOI 10.52202/075280-0802, pp. 18 225–18 250, 2023.

[19] Y. Yang, F. Y. Sun, L. Weihs *et al.*, "Holodeck: Language guided generation of 3D embodied AI environments," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52733.2024.01536, pp. 16 277–16 287, 2024.

[20] Z. Hu, A. Iscen, A. Jain, T. Kipf, and Y. Yue, "SceneCraft: An LLM agent for synthesizing 3D scene as blender code," *arXiv preprint arXiv:2403.01248*, 2024.

[21] X. Ran, Y. Li, L. Xu *et al.*, "Direct numerical layout generation for 3D indoor scene synthesis via spatial reasoning," in *Advances in Neural Information Processing Systems*, vol. 38, DOI 10.52202/085713-4168, pp. 138 615–138 641, 2025.

[22] M. Qi, W. Deng, X. Zhang, and H. Ma, "Global-local monte carlo tree search in vision-language models for text-to-3D indoor scene generation," *arXiv preprint arXiv:2606.06002*, 2026.

[23] L. Gao, J.-M. Sun, K. Mo *et al.*, "SceneHGN: Hierarchical graph networks for 3D indoor scene generation with fine-grained geometry," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, DOI 10.1109/TPAMI.2023.3237577, no. 7, pp. 8902–8919, 2023.

[24] A. Avetisyan, C. Xie, H. Howard-Jenkins *et al.*, "SceneScript: Reconstructing scenes with an autoregressive structured language model," *arXiv preprint arXiv:2403.13064*, 2024.

[25] H. Huang, Y. Chen, Z. Wang *et al.*, "Chat-Scene: Bridging 3D scene and large language models with object identifiers," *arXiv preprint arXiv:2312.08168*, 2024.

[26] Y. Zhang, Z. Li, M. Zhou *et al.*, "The scene language: Representing scenes with programs, words, and embeddings," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52734.2025.02293, pp. 24 625–24 634, 2025.

[27] Y. Yang, B. Jia, P. Zhi *et al.*, "PhyScene: Physically interactable 3D scene synthesis for embodied AI," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52733.2024.01539, pp. 16 262–16 272, 2024.

[28] F. Y. Sun, W. Liu, S. Gu *et al.*, "LayoutVLM: Differentiable optimization of 3D layout via vision-language models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52734.2025.02744, pp. 29 469–29 478, 2025.

[29] N. Pfaff, T. Cohn, S. Zakharov *et al.*, "SceneSmith: Agentic generation of simulation-ready indoor scenes," in *Proceedings of the International Conference on Machine Learning*, 2026, arXiv:2602.09153.

[30] H. I. I. Tam, H. I. D. Pun, A. T. Wang, A. X. Chang, and M. Savva, "SceneEval: Evaluating semantic coherence in text-conditioned 3D indoor scene synthesis," in *IEEE/CVF Winter Conference on Applications of Computer Vision*, DOI 10.1109/WACV61042.2026.00710, pp. 7355–7365, 2026.

[31] Z. Yang, K. Lu, C. Zhang *et al.*, "MMGDreamer: Mixed-modality graph for geometry-controllable 3D indoor scene generation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, DOI 10.1609/aaai.v39i9.33017, no. 9, pp. 9391–9399, 2025.

[32] A. Raistrick, L. Mei, K. Kayan *et al.*, "Infinigen Indoors: Photorealistic indoor scenes using procedural generation," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, DOI 10.1109/CVPR52733.2024.02058, pp. 21 783–21 794, 2024.

[33] R. Fu, Z. Wen, Z. Liu, and S. Sridhar, "AnyHome: Open-vocabulary generation of structured and textured 3D homes," in *Computer Vision – ECCV 2024*, ser. Lecture Notes in Computer Science, DOI 10.1007/978-3-031-72933-1_4, pp. 52–70, 2024, arXiv:2312.06644.

[34] W. Yao, M. R. Min, G. Vosselman, L. E. Li, and M. Y. Yang, "Planner3D: LLM-enhanced graph prior meets 3D indoor scene explicit regularization," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, DOI 10.1109/TPAMI.2025.3602216, pp. 1–17, 2025.

[35] J. Zhang, Z. Zhou, and J. Sun, "Center-aware instance segmentation for close objects in industrial 3-D point cloud scenes," *IEEE Transactions on Industrial Informatics*, vol. 20, DOI 10.1109/TII.2023.3296892, no. 2, pp. 2812–2821, 2024.

[36] Y. Mou, Z. Huang, L. Lin, Y. Guo, and Z. Yang, "Geometry-supervised pose network for accurate retail shelf pose estimation," *IEEE Transactions on Industrial Informatics*, vol. 17, DOI 10.1109/TII.2020.3001147, no. 4, pp. 2357–2364, 2021.

[37] Z. Wang, G. Tian, and T. Liu, "Object-level semantic metric mapping for robot object search in home environment," *IEEE Transactions on Industrial Electronics*, vol. 72, DOI 10.1109/TIE.2024.3472279, no. 5, pp. 5116–5125, 2025.

[38] S. Wang, Y. Tong, and Z. Zhang, "Multi-constraints guided single-view point cloud registration for adaptive robotic manipulation," *IEEE Transactions on Industrial Electronics*, vol. 72, DOI 10.1109/TIE.2024.3508098, no. 8, pp. 8386–8396, 2025.

[39] D. Liu, C. Sun, Z. Gong, J. Yang, X. Liu, and Y. Sun, "Physics-constrained pose refinement of stacked objects via a hierarchical support graph," *IEEE Transactions on Industrial Electronics*, DOI 10.1109/TIE.2026.3677653, pp. 1–11, 2026, early Access.

[40] D. Xing, F. Liu, S. Liu, and D. Xu, "Efficient collision detection and detach control for convex prisms in precision manipulation," *IEEE Transactions on Industrial Informatics*, vol. 14, DOI 10.1109/TII.2018.2816260, no. 12, pp. 5316–5326, 2018.

[41] Z. Pan, Y. Lu, and H. Liu, "MetaFind: Scene-aware 3D asset retrieval for coherent metaverse scene generation," in *Advances in Neural Information Processing Systems*, vol. 38, DOI 10.52202/085713-5306, pp. 176 343–176 366, 2025.

[42] Y. Yang, B. Jia, S. Zhang, and S. Huang, "SceneWeaver: All-in-one 3D scene synthesis with an extensible and self-reflective agent," in *Advances in Neural Information Processing Systems*, vol. 38, DOI 10.52202/085713-4689, pp. 155 685–155 717, 2025.

[43] W. Sun, X. Li, M. Li *et al.*, "Hierarchically-structured open-vocabulary indoor scene synthesis with pre-trained large language model," *arXiv preprint arXiv:2502.10675*, 2025.

[44] W. Feng, H. Zhou, J. Liao *et al.*, "CasaGPT: Cuboid arrangement and scene assembly for interior design," *arXiv preprint arXiv:2504.19478*, 2025.

[45] H. Mao, Y. Huang, J. Lin *et al.*, "VoxScene: Anchor-conditioned voxel diffusion for indoor scene arrangement," *arXiv preprint arXiv:2605.17102*, 2026.

[46] H. Jiang, D. Zhang, D. Weng *et al.*, "HOG-Layout: Hierarchical 3D scene generation, optimization and editing via vision-language models," *arXiv preprint arXiv:2604.10772*, 2026.